

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	1



Programación procedural en SQL

TRIGGERS

Características del SGBD:

- **Versión:** Oracle Database 10g Express Edition
- **Sistema Operativo:** Windows 10

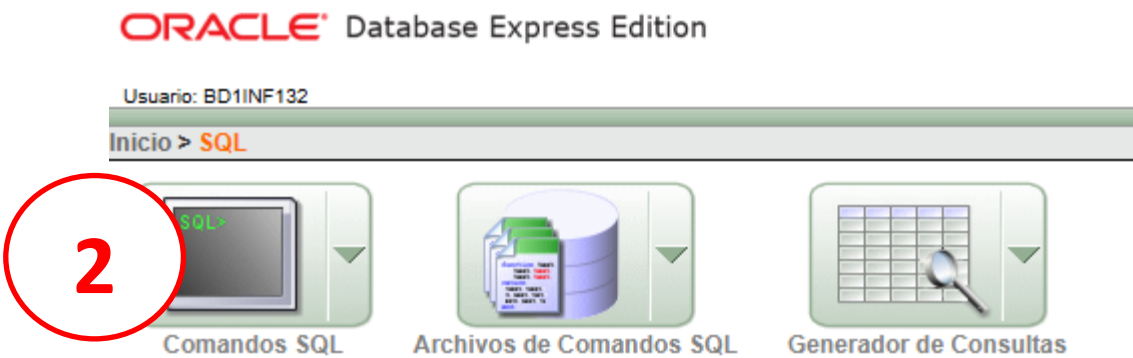
I. Forma de ingresar en el SGBD

Paso 1 → Ingresamos con el usuario que ya tiene la base de datos incorporada:

Paso 2 → Ingresamos a la siguiente opción:

SQL
|___ Comandos SQL

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	2



Paso 3 → La interfaz es la siguiente:



WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	3

II. PARTE TEÓRICA SOBRE TRIGGERS

Definición:

Un *trigger* es un bloque de código PL/SQL que se ejecuta automáticamente cuando ocurre un evento (INSERT, UPDATE o DELETE) sobre una tabla o vista. Técnicamente es un objeto de base de datos que se activa de forma automática como respuesta a una operación DML sobre una tabla. Se ejecuta antes (*BEFORE*) o después (*AFTER*) de una operación DML y puede ser a nivel de fila (*FOR EACH ROW*) o de la tabla completa.

Aplicación e Importancia:

¿Cuándo se aplica?

- Auditoría automática de cambios en registros.
- Validaciones complejas que no pueden ser resueltas con CHECK CONSTRAINTS.
- Mantenimiento automático de registros relacionados (por ejemplo, bitácoras).
- Control de integridad de negocio a nivel de base de datos.
- Prevención de operaciones no deseadas o indebidas.

Importancia:

Permiten que ciertas reglas de negocio se ejecuten automáticamente, garantizando integridad, consistencia y reducción de errores humanos.

Estructura de una transacción

```
CREATE [OR REPLACE] TRIGGER nombre_trigger
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON nombre_tabla
[FOR EACH ROW]
[WHEN (condición)]
DECLARE
    -- Declaración de variables
BEGIN
    -- Cuerpo del trigger
END;
```

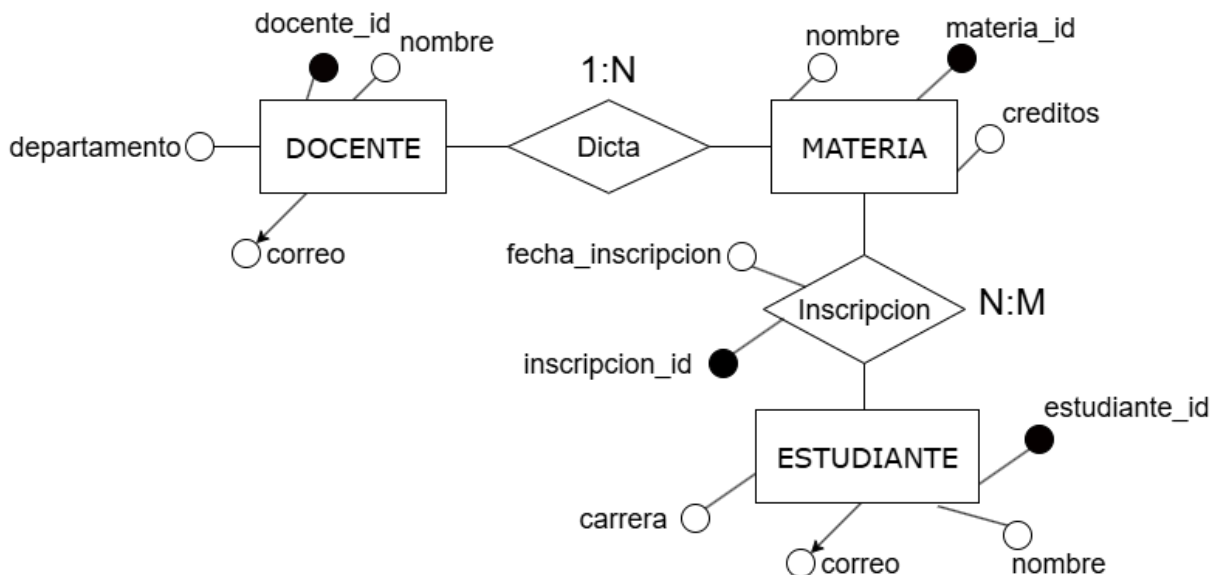
WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	4

Sintaxis:

Elemento	Descripción	Ejemplo
BEFORE / AFTER	Determina si el trigger se ejecuta antes o después del evento DML	AFTER
INSERT, UPDATE, DELETE	Evento que dispara el trigger	AFTER INSERT
FOR EACH ROW	Activa el trigger para cada fila afectada	FOR EACH ROW
:NEW, :OLD	Referencias a los valores nuevos o antiguos	:NEW.nombre, :OLD.correo
Variables	Variables locales al trigger	v_total NUMBER;
Flujo de control	Lógica condicional en el trigger	IF :NEW.nota > 100 THEN ...

II. PARTE PRÁCTICA SOBRE TRIGGERS

Trabajamos con la siguiente base de datos:



WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	5

DOCENTE

DOCENTE_ID	NOMBRE	DEPARTAMENTO	CORREO
1	Ana Torres	Matemáticas	ana.torres@uni.edu
2	Luis Pérez	Historia	luis.perez@uni.edu
3	María López	Literatura	maria.lopez@uni.edu
4	Carlos Gómez	Filosofía	carlos.gomez@uni.edu
5	Sandra Díaz	Informática	sandra.diaz@uni.edu
6	José Fernández	Psicología	jose.fernandez@uni.edu
7	Paula Méndez	Turismo	paula.mendez@uni.edu
8	Andrés Soto	Educación	andres.soto@uni.edu
9	Verónica Ruiz	Lingüística	veronica.ruiz@uni.edu
10	Miguel Romero	Sociología	miguel.romero@uni.edu
11	Elena Castillo	Matemáticas	elena.castillo@uni.edu
12	Fernando Rojas	Historia	fernando.rojas@uni.edu
13	Gabriela Núñez	Literatura	gabriela.nunez@uni.edu
14	Ricardo Salazar	Filosofía	ricardo.salazar@uni.edu
15	Natalia Herrera	Informática	natalia.herrera@uni.edu
16	Diego Vega	Psicología	diego.vega@uni.edu
17	Lorena Castro	Turismo	lorena.castro@uni.edu
18	Raúl Medina	Educación	raul.medina@uni.edu
19	Paola León	Lingüística	paola.leon@uni.edu
20	Javier Ortiz	Sociología	javier.ortiz@uni.edu

20 filas devueltas en 0.00 segundos

[Exportación de CSV](#)

MATERIA

Resultados	Explicar	Describir	SQL Guardado	Historial
MATERIA_ID	NOMBRE	CREDITOS	DOCENTE_ID	
1	Álgebra	4	1	
2	Historia Antigua	3	2	
3	Poesía Moderna	3	3	
4	Ética Profesional	2	4	
5	Programación Básica	5	5	
6	Psicología Infantil	4	6	
7	Geografía Turística	3	7	
8	Pedagogía	4	8	
9	Fonología	2	9	
10	Teoría Social	3	10	
11	Bases de Datos	5	5	
12	Redacción Académica	3	3	
13	Historia Medieval	3	2	
14	Lógica	2	4	
15	Teoría de la Información	4	9	
16	Sistemas de Gestión	5	5	
17	Literatura Latinoamericana	4	3	
18	Neurociencia	3	6	
19	Turismo Cultural	3	7	
20	Políticas Educativas	4	8	

20 filas devueltas en 0.02 segundos

[Exportación de CSV](#)

ESTUDIANTE

ESTUDIANTE_ID	NOMBRE	CARRERA	CORREO
1	Juan Arias	Sistemas	juan.arias@uni.edu
2	Lucía Vidal	Psicología	lucia.vidal@uni.edu
3	Pedro Suárez	Historia	pedro.suarez@uni.edu
4	María Castro	Literatura	maria.castro@uni.edu
5	Andrés León	Educación	andres.leon@uni.edu
6	Carla Núñez	Turismo	carla.nunez@uni.edu
7	Roberto Jiménez	Filosofía	roberto.jimenez@uni.edu
8	Isabel Paredes	Lingüística	isabel.paredes@uni.edu
9	Marco Vela	Sociología	marco.vela@uni.edu
10	Patricia Mendoza	Sistemas	patricia.mendoza@uni.edu
11	Diego Zamora	Historia	diego.zamora@uni.edu
12	Laura Martínez	Psicología	laura.martinez@uni.edu
13	Hugo Luján	Educación	hugo.lujan@uni.edu
14	Daniela Vargas	Literatura	daniela.vargas@uni.edu
15	Cristina Ríos	Lingüística	cristina.rios@uni.edu
16	Mauricio Peña	Filosofía	mauricio.pena@uni.edu
17	Carmen Beltrán	Turismo	carmen.beltran@uni.edu
18	Fabían Ortega	Sociología	fabian.ortega@uni.edu
19	Elena Quintana	Psicología	elena.quintana@uni.edu
20	Antonio Campos	Sistemas	antonio.campos@uni.edu

20 filas devueltas en 0.00 segundos

[Exportación de CSV](#)

INSCRIPCION

INSCRIPCION_ID	ESTUDIANTE_ID	MATERIA_ID	FECHA_INSCRIPCION	NOTA
1	1	1	10-ENE-25	85.5
2	2	6	12-ENE-25	90
3	3	2	15-ENE-25	78
4	4	3	17-ENE-25	88
5	5	8	20-ENE-25	95
6	6	7	22-ENE-25	82
7	7	4	25-ENE-25	78.5
8	8	9	27-ENE-25	88.5
9	9	10	30-ENE-25	80
10	10	5	01-FEB-25	62
11	11	13	03-FEB-25	75
12	12	6	05-FEB-25	85
13	13	8	08-FEB-25	91
14	14	17	10-FEB-25	89
15	15	15	12-FEB-25	84.5
16	16	14	15-FEB-25	79
17	17	19	18-FEB-25	87
18	18	20	20-FEB-25	90.5
19	19	18	22-FEB-25	88
20	20	16	25-FEB-25	93

20 filas devueltas en 0.00 segundos

[Exportación de CSV](#)

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	6

Ejemplo 1: Auditoría de inserción de estudiantes

Descripción: Registra en consola cuando se inserta un nuevo estudiante.

```
CREATE OR REPLACE TRIGGER trg_log_insert_estudiante
AFTER INSERT ON ESTUDIANTE
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Nuevo estudiante insertado: ' ||
:NEW.nombre);
END;
```

Para probar:

```
INSERT INTO ESTUDIANTE (estudiante_id, nombre, correo, carrera)
VALUES (1001, 'Luis Torres', 'luis.torres@email.com',
'Psicología');
```

Resultado esperado:

Nuevo estudiante insertado: Luis Torres

Notas:

- Si estás usando SQL*Plus o SQL Developer, activa la salida: **SET SERVEROUTPUT ON**; Esto permite que puedas ver el mensaje generado por DBMS_OUTPUT.PUT_LINE.

Ejemplo 2: Validar notas máximas al insertar en INSCRIPCION

Descripción: Impide que se inserte una nota mayor a 100.

```
CREATE OR REPLACE TRIGGER trg_validar_nota
BEFORE INSERT ON INSCRIPCION
FOR EACH ROW
BEGIN
    IF :NEW.nota > 100 THEN
        RAISE_APPLICATION_ERROR(-20001, 'La nota no puede ser mayor
a 100');
```

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	7

```
END IF;
END;
```

Para probar:

1)

```
INSERT INTO INSCRIPCION (inscripcion_id, estudiante_id,
materia_id, nota, fecha_inscripcion)
VALUES (1, 101, 201, 95, TO_DATE('2025-07-10', 'YYYY-MM-DD'));
```

Resultado esperado:

Este INSERT se ejecutará sin errores.

2)

```
INSERT INTO INSCRIPCION (inscripcion_id, estudiante_id,
materia_id, nota, fecha_inscripcion)
VALUES (2, 101, 201, 110, TO_DATE('2025-07-10', 'YYYY-MM-DD'));
```

Resultado esperado:

```
ORA-20001: La nota no puede ser mayor a 100
ORA-06512: en "TRG_VALIDAR_NOTA", línea 3
ORA-04088: error durante la ejecución del trigger
'TRG_VALIDAR_NOTA'
```

Notas:

- Si estás usando SQL*Plus o SQL Developer, activa la salida: **SET SERVEROUTPUT ON**; Esto permite que puedas ver el mensaje generado por DBMS_OUTPUT.PUT_LINE.
- RAISE_APPLICATION_ERROR permite que un desarrollador **detenga la ejecución de un bloque PL/SQL** y muestre un **mensaje de error definido por el usuario**, asignándole un **código de error específico** (entre -20000 y -20999).
- **-20001** Es un código de error definido por el usuario. Oracle permite lanzar errores personalizados entre -20000 y -20999.

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	8

Ejemplo 3: Bitácora de cambios de materia

Descripción: Muestra los datos antes y después de un UPDATE sobre MATERIA.

```
CREATE OR REPLACE TRIGGER trg_log_update_materia
BEFORE UPDATE ON MATERIA
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Materia actualizada de ' || :OLD.nombre
    || ' a ' || :NEW.nombre);
END;
```

Para probar:

```
UPDATE MATERIA
SET nombre = 'Matemáticas Avanzadas'
WHERE materia_id = 101;
```

Resultado esperado:

Materia actualizada de Matemáticas a Matemáticas Avanzadas

Notas:

- El trigger se ejecuta **antes** de cada fila actualizada (FOR EACH ROW).
- El mensaje muestra el nombre antiguo (:OLD.nombre) y el nuevo (:NEW.nombre).

Ejemplo 4: Prohibir eliminación de docentes con materias asignadas

Descripción: Evita eliminar docentes que tienen materias asignadas.

```
CREATE OR REPLACE TRIGGER trg_prevent_delete_docente
BEFORE DELETE ON DOCENTE
FOR EACH ROW
DECLARE
    v_count NUMBER;
```


WORKBOOK 8

Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	9

```
BEGIN
  SELECT COUNT(*) INTO v_count
  FROM MATERIA
  WHERE docente_id = :OLD.docente_id;
  IF v_count > 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'No se puede eliminar:
tiene materias asignadas');
  END IF;
END;
```

Para probar:

```
1)
-- Inserta un docente sin materias
INSERT INTO DOCENTE (docente_id, nombre, departamento)
VALUES (500, 'Prueba Sin Materias', 'Lenguas');

-- Intenta eliminarlo
DELETE FROM DOCENTE WHERE docente_id = 500;
```

Resultado esperado:

Resultado esperado: se elimina correctamente.

```
2)

-- Inserta un docente con materia
INSERT INTO DOCENTE (docente_id, nombre, departamento)
VALUES (501, 'Docente Asignado', 'Historia');

-- Inserta una materia con ese docente
INSERT INTO MATERIA (materia_id, nombre, creditos, docente_id)
VALUES (301, 'Historia Moderna', 3, 501);

-- Intenta eliminar el docente
DELETE FROM DOCENTE WHERE docente_id = 501;
```

Resultado esperado:

Error ORA-20002: No se puede eliminar: tiene materias asignadas

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	10

Notas:

- Para pruebas seguras, usa una **transacción controlada**:

```

BEGIN
    DELETE FROM DOCENTE WHERE docente_id = 501;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE (SQLERRM) ;
END ;

```

- En un trigger **BEFORE DELETE**, **:OLD** se refiere al **registro que está por eliminarse**.
:OLD.docente_id accede al **docente_id** de ese registro.

WORKBOOK 8		
Materia	Docente:	Número de Página:
INF-272 Taller de base de datos	M.Sc. Augusto Cesar Lunasco Cusi	11

III. PRÁCTICA

1. Registrar automáticamente la fecha actual al insertar un nuevo docente.
2. Prevenir eliminación de materias si tienen inscripciones activas.
3. Validar que la nota no sea mayor a 100.
4. Asignar fecha de inscripción automáticamente.
5. Mostrar en consola las materias insertadas.
6. Impedir inserción de estudiantes sin carrera.
7. Validar que los docentes tengan correo electrónico.
8. Auditar actualizaciones de notas mostrando cambios.
9. Limitar a 5 materias por docente.
10. Validar formato de correo de estudiante.
11. Prevenir eliminación de estudiantes con inscripciones.
12. Evitar insertar materias sin nombre.
13. Auditar eliminaciones de docentes.
14. Validar que las notas no sean negativas.
15. Bloquear modificación de carrera una vez registrada.
16. Validar que el correo del docente no esté duplicado.
17. Auditar cambios en la carrera del estudiante.
18. Registrar la hora exacta de inscripción.
19. Validar que los créditos de materia estén entre 1 y 5.
20. Evitar inscripciones duplicadas.

Nota: Para los ejercicios de auditoria debe crear una tabla que registre el DML correspondiente.